Deep Q-Network Control of the Unitree G1 Left Elbow

Technical Report

Course: CSCN8020 - Reinforcement Learning
Assignment: DQN Assignment
Student: Ali Cihan Ozdemir - 9091405
Instructor: Prof. Enrique Espinosa
Institution: Conestoga College, Ontario, Canada

---

1. Introduction and Connection to the G1 Primer Workshop

This assignment builds directly upon the groundwork laid in the **Unitree MuJoCo G1 Primer Workshop**. In the primer, we analyzed the biomechanical structure of the Unitree G1 humanoid robot's left arm, specifically the 29 Degrees of Freedom (DoF) model. We verified the robot's physical assets and established a fixed-base simulation model (`assets/g1_fixed_base/scene_29dof_fixed_base.xml`) where the pelvis is anchored at a height of 0.80 meters. This fixed-base setup eliminates complex full-body balance issues, providing a stable platform to study the control of the left elbow joint (`left_elbow_joint`).

While the primer workshop demonstrated manual joint control using a proportional-derivative (PD) controller and a rule-based script, this assignment replaces the hand-crafted rule-based heuristic with a self-learning **Deep Q-Network (DQN)** agent. The DQN learns an optimal control policy by interacting with the environment to dynamically adjust the joint target, optimizing task performance directly from state observations.

---

2. Environment Observation, Action, Reward, and Success Definitions

The control task is framed as a Markov Decision Process (MDP) using the custom Gymnasium environment `G1ElbowTargetEnv` . The key components of the environment are:

2.1 Observation Space

The state observation is a continuous vector $s_t \in \mathbb{R}^4$ representing the state of the joint relative to the current target:

$$s_t = \begin{bmatrix} \theta_t \\ \dot{\theta}_t \\ g \\ g - \theta_t \end{bmatrix}$$ where:

- θ_t: Current physical elbow joint angle in radians.
- $\dot{\theta}_t$: Current elbow joint velocity in radians/second.
- g: Target goal angle in radians (sampled from $[-0.8, +0.8]$ during training).
- $g - \theta_t$: Current joint position error.

2.2 Action Space

The agent makes high-level decisions using a discrete action space $\mathcal{A} = \{0, 1, 2\}$, which modifies the internal controller target θ_{target} at each environment step:

- **Action 0 (Decrease):** $\theta_{target} \leftarrow \theta_{target} - \Delta\theta$
- **Action 1 (Hold):** $\theta_{target} \leftarrow \theta_{target}$ (no change)
- **Action 2 (Increase):** $\theta_{target} \leftarrow \theta_{target} + \Delta\theta$

Here, the step increment is fixed at $\Delta\theta = 0.08$ radians. The updated θ_{target} is clipped to the physical joint range: $[-0.9, 1.5]$ radians.

2.3 Reward Function

The reward $R(s_t, a_t)$ is designed to guide the agent toward the goal while promoting joint stability: $R(s_t, a_t) = -|g - \theta_t| + R_{bonus} - R_{penalty}$ where:

- **Goal proximity penalty:** $-|g - \theta_t|$ forces the agent to minimize the absolute joint error.
- **Success bonus:** $R_{bonus} = +1.0$ if the joint error is within the success tolerance ($|g - \theta_t| \leq 0.04$ rad).
- **Control stability penalty:** $R_{penalty} = 0.05$ is applied if the agent is inside the success zone but chooses to actuate (Action 0 or 2) instead of holding (Action 1). This penalizes unnecessary target changes.
- **Termination reward:** A large terminal bonus of $+10.0$ is awarded upon episode completion if the success condition is met.

2.4 Success and Termination Conditions

- **Terminated:** An episode is terminated successfully if the elbow remains within the success tolerance ($|g - \theta_t| \leq 0.04$ rad) for a consecutive success streak of at least 8 steps.
- **Truncated:** An episode is truncated if the step count reaches the maximum episode length of 150 steps without completing the success condition.

---

3. Final Q-Network Architecture

The Q-network approximates the action-value function $Q(s, a; \theta)$, mapping states to Q-values for each discrete action.

```
Input Vector (4) ---> Linear(4 -> 64) ---> ReLU ---> Linear(64 -> 64) ---> ReLU ---> Linear(64 -> 3) ---> Output Q-Values (3)
```

The network structure is defined as follows:

- **Input Layer:** 4 units representing the state vector s_t.
- **Hidden Layer 1:** Fully-connected layer (Linear) mapping $4 \to 64$ units, followed by a Rectified Linear Unit (**ReLU**) activation function.
- **Hidden Layer 2:** Fully-connected layer (Linear) mapping $64 \to 64$ units, followed by a **ReLU** activation function.
- **Output Layer:** Fully-connected layer (Linear) mapping $64 \to 3$ units representing the unconstrained Q-values: $[Q(s, 0), Q(s, 1), Q(s, 2)]$.

- **Weight Initialization: Kaiming Normal** initialization is used for weights in linear layers to stabilize backpropagation through the ReLU units, with biases initialized to 0.

---

4. Replay-Buffer and Target-Network Methodology

To stabilize learning and break the temporal correlation of sequential experiences, two core DQN mechanisms are implemented:

4.1 Experience Replay Buffer

A replay buffer $\mathcal{D}$ of capacity $N = 50,000$ transitions is allocated. The agent stores transition tuples $(s_t, a_t, r_t, s_{t+1}, \text{terminated}_t)$ at each step. During training, random mini-batches of size $B = 64$ are sampled from the buffer. This off-policy sampling breaks sequential correlations, making the training data independent and identically distributed (i.i.d.), which is a requirement for standard gradient descent optimization.

4.2 Target Network

To prevent divergence caused by moving targets during value iteration, we maintain two separate networks:

1. **Online Q-Network (Q):** Parameterized by weights θ. Optimized at every step.
2. **Target Q-Network ($\hat{Q}$):** Parameterized by weights θ^-. Kept stationary and used only to compute target Q-values.

The target network is synchronized with the online network using a **hard update** every 250 optimization steps: $\theta^- \leftarrow \theta$

---

5. Bellman Target and Loss Formulation

The agent optimizes the parameters θ of the online network by minimizing the temporal-difference (TD) loss over a sampled mini-batch of B transitions: $\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^{B} \mathcal{L}_{Huber} \left(Q(s_i, a_i; \theta) - y_i \right)$

The TD target y_i is computed using the target network $\hat{Q}$: $y_i = r_i + \gamma(1 - \text{terminated}_i) \max_{a'} \hat{Q}(s'_i, a'; \theta^-)$ where:

- $\gamma = 0.95$ is the temporal discount factor.
- terminated_i is the success termination flag. If the transition leads to a true terminal state, the target reduces to $y_i = r_i$, preventing bootstrapping from terminal states.
- **Truncation Treatment:** When the episode ends due to the maximum step limit (150 steps), terminated_i is `False` but `truncated` is `True`. In this case, we **do bootstrap** because truncation is a time limit and not a terminal state. Setting `terminated` to `False` allows the target network to correctly bootstrap the remaining value of the state.

The loss function is the **Huber Loss** (Smooth L1 Loss), which behaves like Mean Squared Error (MSE) for small errors and like Mean Absolute Error (MAE) for large errors, reducing the sensitivity to outliers and exploding gradients:

$$\mathcal{L}_{Huber}(x) = \begin{cases} \frac{1}{2}x^2 & \text{if } |x| \leq 1 \\ |x| - \frac{1}{2} & \text{otherwise} \end{cases}$$

Gradient clipping is applied to clip the norm of the gradients at 1.0 for further stability.

6. Exploration Strategy

To balance exploration (learning about the environment) and exploitation (using what has been learned), an **ϵ-greedy** exploration strategy is used. The probability of choosing a random action is ϵ, while the probability of choosing the greedy action ($\arg\max_a Q(s, a)$) is $1 - \epsilon$.

Epsilon starts at $\epsilon_{start} = 1.00$ and decays after each episode according to: $\epsilon_{t+1} = \max(\epsilon_{min}, \epsilon_t \times d)$ where $\epsilon_{min} = 0.05$ is the floor. We perform a parameter study comparing two decay rates d:

- **Configuration A (Baseline):** $d = 0.995$ (longer exploration phase).
- **Configuration B (Faster Decay):** $d = 0.985$ (earlier exploitation phase).

---

7. Training Methodology and Reproducibility Controls

Training was executed headlessly to eliminate rendering overhead.

- **Reproducibility:** Seeds were explicitly set to 42 for `random`, `numpy`, `torch`, and the Gymnasium environment.
- **Hardware Acceleration:** PyTorch was configured to use **Apple Silicon MPS (Metal Performance Shaders)** on the MacBook Pro M1, accelerating tensor computations on the M1 GPU.
- **Warm-up:** Optimization only begins after the replay buffer has accumulated at least 500 transitions.
- **Training duration:** Both configurations were trained for 600 episodes.

---

8. Results for Both Epsilon-Decay Configurations

The training metrics for both configurations are summarized below:

Metric	Configuration A (Decay = 0.995)	Configuration B (Decay = 0.985)
Total Training Episodes	600	600
Wall-Clock Training Time	161.10 seconds	128.77 seconds
Final Epsilon	0.0500	0.0500
Mean Reward (Final 20 Training Episodes)	10.73	10.84
Training Success Rate (Final 50 Episodes)	100.0%	100.0%

Both configurations successfully converged to a 100.0% success rate in training. Configuration B trained slightly faster (128.77s vs 161.10s) because it transitioned to exploitation earlier, resulting in shorter episodes (due to faster termination) during mid-training.

---

9. Required Plots and Evaluation Tables

9.1 Training Performance Plots

The training metrics were logged and saved to the following folders:

- Configuration A plots: results/config_a/
- Configuration B plots: results/config_b/

The overlays comparing reward and success rates between the configurations are saved in the root results folder:

- Epsilon Decay Reward Comparison
- Epsilon Decay Success Comparison

9.2 Final Evaluation Tables (Greedy Policy, $\epsilon = 0.0$)

Configuration A Evaluation:

- Overall Successes: 20/20 (100.0% success rate)
- Overall Mean Reward: 13.32

Goal (rad)	Episodes	Successes	Success Rate	Mean Reward
-0.8 rad	5	5	100.0%	11.06
-0.4 rad	5	5	100.0%	15.57
+0.4 rad	5	5	100.0%	15.64
+0.8 rad	5	5	100.0%	11.02
Overall	20	20	100.0%	13.32

Configuration B Evaluation:

- Overall Successes: 20/20 (100.0% success rate)
- Overall Mean Reward: 13.16

Goal (rad)	Episodes	Successes	Success Rate	Mean Reward
-0.8 rad	5	5	100.0%	10.98

Goal (rad)	Episodes	Successes	Success Rate	Mean Reward
-0.4 rad	5	5	100.0%	15.44
+0.4 rad	5	5	100.0%	15.36
+0.8 rad	5	5	100.0%	10.87
Overall	20	20	100.0%	13.16

---

10. Comparison with the Rule-Based Baseline

The greedy evaluation results of both DQN agents were compared against the hand-written rule-based baseline policy over the same 20 benchmark episodes:

Metric	Rule-Based Policy	Selected DQN (Config A)
Successes / 20	20 / 20	20 / 20
Success Rate	100.0%	100.0%
Mean Cumulative Reward	12.87	13.32
Mean Episode Length (steps)	24.0	19.8
Mean Final Absolute Error (rad)	0.0122	0.0039
Main Qualitative Behaviour	Moves target linearly; holds target within static error.	Accelerates target dynamically; fine-tunes near goal for high precision.

Discussion Questions:

1. **Which policy is more sample efficient?** The hand-written rule-based policy is more sample-efficient during *development*, requiring 0 environment interactions to define. However, during *execution*, the DQN policy is more step-efficient, completing the task in **19.8 steps** compared to the rule-based policy's **24.0 steps**.
2. **Which policy is more stable near the goal?** The DQN policy is significantly more stable and precise near the goal, achieving a mean absolute error of **0.0039 rad** compared to the rule-based policy's **0.0122 rad**.
3. **Does the DQN generalize across all four target angles?** Yes, both DQN configurations successfully generalized to all four unseen/benchmark target angles (`−0.8, −0.4, +0.4, +0.8` rad), achieving a perfect 100.0% success rate on each.

4. **Does the DQN learn to use HOLD appropriately?** Yes. Because of the reward penalty ($R_{penalty} = -0.05$) for selecting non-HOLD actions when inside the success zone, the DQN learned to select Action 1 (HOLD) to avoid the penalty, allowing the low-level PD controller to settle the joint.
5. **Are there signs of oscillation or unnecessary target changes?** No. The reward penalty for actions inside the success zone successfully eliminated target oscillation.
6. **Why might a hand-written policy outperform a learned policy in this simple task?** For extremely simple single-joint systems, a hand-written policy does not require any training computation, training data, or hyperparameter optimization. It behaves predictably and is transparent, whereas neural networks behave as a black box and can act unpredictably on out-of-distribution inputs.

---

11. Discussion of Failures, Oscillation, Stability, and Generalization

During the early phases of training, the DQN agent struggled to stabilize. Epsilon was high, leading to random jittering and joint oscillations. As epsilon decayed, the agent learned the mapping from the joint error ($g - \theta$) to target increments.

By introducing the stability penalty for non-HOLD actions in the success zone, we successfully eliminated the target oscillation that often plagues discrete control agents. In terms of generalization, training on a continuous goal range of $[-0.8, +0.8]$ rad allowed the network to learn a smooth, interpolating policy. The network successfully generalized to the four evaluation goals without overfitting.

---

12. Evidence-Based Recommendation of the Better Exploration-Decay Setting

We select **Configuration A (decay rate = 0.995)** as the superior configuration. While Configuration B trained slightly faster (128.77s vs 161.10s) due to rapid exploration decay, Configuration A achieved a higher overall mean reward (**13.32** vs **13.16**) and maintained greater stability during evaluation. The longer exploration period of Configuration A allowed it to sample more diverse states and learn a slightly more optimal value function, resulting in cleaner control actions near the target boundaries.

---

13. Limitations and Proposed Future Improvements

The current approach has several limitations:

1. **Discretized Actions:** The step size $\Delta\theta = 0.08$ rad is fixed. A smaller step size would allow even higher precision but would increase the time to reach the goal. A continuous control method (such as DDPG or SAC) would allow the agent to output continuous target changes, achieving smooth and optimal joint control.
2. **Fixed Base Limitation:** The model assumes a fixed base. In a full G1 robot walking or running, elbow movements affect the robot's overall momentum and balance. Future work should integrate the elbow controller with full-body dynamics.
3. **Domain Randomization:** The simulator has fixed physical parameters (friction, mass). To deploy the policy on a real G1 physical robot, we should apply domain randomization (varying joint damping, friction, and mass) during training to make the policy robust to physical discrepancies.